

Nihai mimari özeti:

1) Problem tanımı

- Amaç: t+1 adımında **yukarı mı / aşağı mı** hareket olacağını tahmin etmek
- Problem tipi: **ikili sınıflandırma**
- Etiketler:
 - 1 = bull
 - 0 = bear

2) Veri yapısı

Dosyalardan çıkan mevcut yapı:

- X_tensor.npy: (2572, 21, 7)
- y_tensor.npy: (2572,)
- Ana feature seti:
 - VIX_Term_Structure
 - Yield_Curve
 - SKEW_Index
 - Risk_Appetite_Ratio
 - Crude_Oil
 - VWAP_Deviation
 - Volume_Momentum

3) Label üretimi

Getiri:

$$rt+1 = \text{Close}_{t+1} - \text{Close}_t$$

Etiket:

- $r_{t+1} > +thr \rightarrow 1$
- $r_{t+1} < -thr \rightarrow 0$
- aradaki bölge $\rightarrow$ **neutral / ignore**

Kural:

- threshold **öğrenilen model parametresi değil**
- threshold sadece **train kısmından** hesaplanır
- val/testte aynı sabit threshold uygulanır

4) Split stratejisi

- Veri **kronolojik** kullanılacak
- Random split yok
- Shuffle yok
- **Son %15** = final holdout test
- İlk %85 = development alanı

Development içinde:

- **4 fold expanding window validation**

Yani:

- train fold'dan fold'a büyür
- val her zaman train'in ilerisindeki zaman bloğu olur

5) Sequence hizalaması

- lookback hiperparametre olacak
- ilk deney varsayılanı: **21**

- giriş: [t-20, ..., t]
- hedef: t+1 yönü

Yani model geçmiş 21 adımı görüp bir sonraki adımın bull/bear olmasını tahmin eder.

6) Preprocessing kuralları

- veri tarih sırasına göre işlenecek
- NaN yönetimi:
 - önce **forward fill**
 - başta kalan NaN varsa **drop**
 - backfill yok
- leakage yasak

7) Scaling

Scaling fold bazlı yapılacak:

- scaler sadece **fold train** üzerinde fit edilir
- val/test sadece transform edilir

Scaler stratejisi:

- StandardScaler
- RobustScaler

İlk sistem bunları karşılaştırabilecek şekilde modüler kurulacak.

8) Model stratejisi

İlk deney seti:

- **Logistic Regression**
- **GRU**

Kod mimarisi modüler olacak, böylece sonuç yetersizse aynı pipeline içinde:

- **1D CNN**
eklenebilecek

Yani sistem:

- baseline model
- sequence model
- sonra gerekirse yeni model
eklemeye uygun olacak

9) Class imbalance

- oversampling yok
- class weight kullanılacak

Strateji:

- Logistic Regression için: class_weight="balanced"
- GRU/CNN için: **weighted loss**

10) Eğitim kuralı

Neural modeller için:

- max_epochs = 100
- early_stopping_patience = 10
- monitor = **validation MCC**
- best checkpoint restore

Model seçimi:

- fold'lar üzerindeki **ortalama MCC**

11) Değerlendirme metrikleri

Ana metrik:

- **MCC**

Yardımcı metrikler:

- **F1**
- **Balanced Accuracy**

Ek çıktı:

- confusion matrix

12) Karar eşiği

Ayrı bir decision threshold olacak:

- ilk deneyde 0.50
- ama config'te parametre olarak tutulacak
- sonra validation ile tune edilebilir

13) Feature stratejisi

İlk sürüm:

- mevcut **7 feature** ile başlanacak

Sonraki aşama:

- feature engineering ayrı modülden eklenecek
- config ile açılıp kapanabilecek

14) Modüler tasarım

Sabit kalan parçalar:

- chronological split
- threshold üretimi
- preprocessing
- scaler akışı
- evaluation
- logging

Değişebilir parçalar:

- model
- scaler
- lookback
- threshold stratejisi
- decision threshold
- feature set

Yani yapı "lego" gibi olacak.

15) Config yönetimi

Seçilen yaklaşım:

- **YAML + override**

Böylece:

- varsayılan ayarlar düzenli saklanır
- küçük değişiklikler kolay yapılır
- deney tekrar üretilebilir olur

16) Kod organizasyonu

Başlangıç yaklaşımı:

- tamamen tek dosya değil
- gereksiz aşırı parçalı da değil
- orta düzey modüler yapı

Temel modüller:

- data loading
- preprocessing
- labeling
- splitter

- sequence builder
- model factory
- trainer
- metrics/evaluation
- experiment runner

17) Logging ve kayıt

Her deney için saklanacak:

- config snapshot
- model adı
- scaler
- lookback
- threshold yöntemi/değeri
- fold bazlı skorlar
- ortalama/std metrikler
- final test skorları
- confusion matrix
- prediction çıktıları

Format:

- detaylı kayıt: **JSON**
- deney özeti: **CSV**
- model checkpoint: dosya

18) Reproducibility

- tek bir global **seed**
- config içinde saklanacak
- tüm pipeline buna göre çalışacak

İleride istenirse multi-seed deneylerine genişletilebilir.

19) Final test disiplini

Final test:

- model seçimi için kullanılmayacak
- threshold seçimi için kullanılmayacak
- scaler seçimi için kullanılmayacak
- hyperparameter tuning için kullanılmayacak

Sadece en sonda **bir kez** gerçek değerlendirme için kullanılacak.

Model İskeleti

6 Nisan 2026 Pazartesi

02:20

1. Uygulama sırası

Faz 1 — Veri katmanı

Önce tek bir giriş noktası kurulur:

- CSV yüklenir

- tarih sütunu parse edilir
- veri kronolojik sıralanır
- ffill uygulanır
- başta kalan NaN satırlar silinir
- yalnızca seçilen feature kolonları ve hedef için gereken fiyat kolonu tutulur

Çıktı:

- temiz DataFrame

Faz 2 — Final test ayrımı

Veri en başta ikiye ayrılır:

- ilk %85 → development
- son %15 → final test

Bu aşamadan sonra final test hiçbir seçimde kullanılmaz.

Çıktı:

- dev_df
- test_df

Faz 3 — Walk-forward fold üretimi

dev_df içinde expanding-window fold'lar üretilir.

Örnek mantık:

- Fold 1: train erken blok, val sonraki blok
- Fold 2: train büyür, val sonraki blok
- Fold 3: train daha da büyür, val sonraki blok
- Fold 4: train daha da büyür, val sonraki blok

Çıktı:

- fold index listesi

Faz 4 — Her fold içinde label threshold hesaplama

Her fold'da yalnızca train kısmından:

- $\text{Close}[t+1]$ ve $\text{Close}[t]$ ile getiri hesaplanır
- $\text{abs}(\text{return})$ dağılımından threshold seçilir
- bu tek sayı sabitlenir

Sonra aynı threshold:

- train label üretimi
- val label üretimi

için kullanılır.

Çıktı:

- `threshold_value`
- `y_train, y_val`

Faz 5 — Sequence üretimi

Lookback config'ten gelir.

Örnek:

- `lookback = 21`

Her sample:

- `input = [t-lookback+1, ..., t]`
- `target = t+1` yön etiketi

Neutral bölgeye düşen örnekler çıkarılır.

Çıktı:

- sequence tensor veya flattened matrix

Faz 6 — Scaling

Her fold'da:

- scaler yalnızca train feature'larında fit edilir

- val aynı scaler ile transform edilir

Model tipine göre iki format tutulur:

- tabular model için: flatten edilmiş X
- sequence model için: 3D X

Çıktı:

- X_train_model_ready
- X_val_model_ready

Faz 7 — Model eğitimi

İlk etapta iki model:

- Logistic Regression
- GRU

Her model aynı fold'lar üzerinde çalışır.

Neural model için:

- weighted loss
- early stopping
- best checkpoint restore

Çıktı:

- fold bazlı metrikler
- model artifact

Faz 8 — CV değerlendirme

Her deney için:

- fold MCC
- fold F1
- fold Balanced Accuracy

sonra:

- mean

- std

hesaplanır.

Model seçimi:

- ortalama MCC'ye göre

Faz 9 — En iyi config ile yeniden eğitim

Seçilen config ile:

- development alanının tamamı kullanılır
- threshold development-train mantığıyla hesaplanır
- scaler fit edilir
- model yeniden eğitilir

Faz 10 — Final test

Son %15 blokta:

- aynı threshold
- aynı scaler
- aynı model

ile tek seferlik değerlendirme alınır.

Rapor:

- MCC
- F1
- Balanced Accuracy
- confusion matrix
- prediction CSV

2. Klasör yapısı

project/

```
└─ configs/
|   └─ base.yaml
|   └─ model_logreg.yaml
|   └─ model_gru.yaml
|   └─ model_cnn1d.yaml
|
└─ data/
|   └─ raw/
|   └─ processed/
|
└─ src/
|   └─ data/
|       └─ load_data.py
|       └─ preprocess.py
|       └─ labeling.py
|       └─ sequence_builder.py
|       └─ splitters.py
|       |
|   └─ models/
|       └─ logistic_regression.py
|       └─ gru_model.py
|       └─ cnn1d_model.py
|       └─ model_factory.py
|       |
|   └─ training/
|       └─ trainer.py
```

```
| | └─ losses.py
| | └─ callbacks.py
| |
| └─ evaluation/
| | └─ metrics.py
| | └─ reports.py
| | └─ plots.py
| |
| └─ utils/
| | └─ config.py
| | └─ seed.py
| | └─ io.py
| |
| └─ run_experiment.py
|
└─ artifacts/
  └─ models/
  └─ metrics/
  └─ predictions/
  └─ plots/
  └─ experiments_summary.csv
```

3. Dosya bazlı sorumluluklar

src/data/load_data.py

Görev:

- CSV oku
- tarih parse et
- kronolojik sırala

Fonksiyonlar:

- `load_market_data(path, date_col) -> pd.DataFrame`

src/data/preprocess.py

Görev:

- `ffill`
- baştaki NaN temizliği
- kolon seçimi

Fonksiyonlar:

- `apply_missing_value_policy(df, method="ffill")`
- `select_columns(df, feature_cols, close_col)`

src/data/labeling.py

Görev:

- forward return hesaplama
- threshold seçimi
- bull/bear/neutral etiketleme

Fonksiyonlar:

- `compute_forward_return(close_series, horizon=1)`
- `compute_threshold(train_returns, method, quantile)`
- `make_labels(returns, threshold)`

Not:

- threshold her fold'da sadece train'den hesaplanacak

src/data/sequence_builder.py

Görev:

- lookback'e göre pencere üretmek
- target hizalaması yapmak
- neutral örnekleri elemek

Fonksiyonlar:

- build_sequences(df, feature_cols, labels, lookback)
- flatten_sequences(X_seq)

src/data/splitters.py

Görev:

- final holdout split
- expanding-window fold üretmek

Fonksiyonlar:

- split_dev_test(df, test_ratio)
- make_expanding_folds(n_samples, n_folds, val_ratio)

src/models/logistic_regression.py

Görev:

- sklearn logistic model wrapper

Fonksiyonlar / sınıf:

- build_logreg(config)

src/models/gru_model.py

Görev:

- PyTorch GRU sınıfı

Sınıf:

- GRUClassifier(nn.Module)

İçerik:

- input size = feature count
- hidden dim config'ten
- dropout config'ten
- output = sigmoid/logit

src/models/cnn1d_model.py

Görev:

- ileride kolay eklenebilir CNN modeli

Sınıf:

- CNN1DClassifier(nn.Module)

Başta boş bırakılabilir ya da stub olabilir.

src/models/model_factory.py

Görev:

- model adına göre doğru modeli üretmek

Fonksiyon:

- build_model(model_name, config, input_shape)

Bu sayede:

- logreg
- gru
- cnn1d

tek yerden seçilir.

src/training/losses.py

Görev:

- weighted BCE / CE üretmek

Fonksiyon:

- `build_loss(config, class_weights=None)`

src/training/callbacks.py

Görev:

- early stopping

Sınıf:

- `EarlyStopping`

src/training/trainer.py

Görev:

- model training loop
- validation metric takibi
- checkpoint kaydı

Fonksiyonlar:

- `train_sklearn_model(...)`
- `train_torch_model(...)`
- `predict_proba(...)`

src/evaluation/metrics.py

Görev:

- MCC
- F1
- Balanced Accuracy
- confusion matrix

Fonksiyonlar:

- `compute_classification_metrics(y_true, y_pred, y_prob=None)`

src/evaluation/reports.py

Görev:

- fold sonuçlarını toparlamak
- JSON yazmak
- CSV özet satırı üretmek

Fonksiyonlar:

- save_experiment_report(...)
- append_summary_row(...)

src/utils/config.py

Görev:

- YAML yüklemek
- override uygulamak

Fonksiyonlar:

- load_config(path, overrides=None)

src/utils/seed.py

Görev:

- global seed ayarlamak

Fonksiyon:

- set_global_seed(seed)

src/utils/io.py

Görev:

- klasör oluşturma
- json/csv yazma
- artifact path yönetimi

src/run_experiment.py

Görev:

- tüm pipeline'ın ana orkestrasyonu

Yapacağı işler:

1. config yükle
2. seed ayarla
3. veriyi yükle
4. preprocess uygula
5. dev/test ayır
6. CV fold'larını oluştur
7. model adaylarını çalıştır
8. en iyi config'i seç
9. dev üzerinde yeniden eğit
10. testte değerlendir
11. artifact'leri yaz

4. Config yapısı

configs/base.yaml yaklaşık şöyle olmalı:

experiment:

name: baseline_bull_bear

seed: 42

data:

file_path: data/raw/market_data.csv

date_col: Date

close_col: Nasdaq_Close

feature_cols:

- VIX_Term_Structure
- Yield_Curve
- SKEW_Index
- Risk_Appetite_Ratio
- Crude_Oil
- VWAP_Deviation
- Volume_Momentum

split:

final_test_ratio: 0.15

n_folds: 4

val_ratio_within_dev: 0.10

method: expanding

labeling:

horizon: 1

threshold_method: quantile

threshold_quantile: 0.40

neutral_policy: drop

sequence:

lookback: 21

preprocessing:

missing_policy: ffill_then_drop_head

scalers:

- standard

- robust

decision:

probability_threshold: 0.50

training:

batch_size: 64

max_epochs: 100

early_stopping_patience: 10

monitor_metric: mcc

learning_rate: 0.001

evaluation:

primary_metric: mcc

secondary_metrics:

- f1

- balanced_accuracy

models:

enabled:

- logreg

- gru

logreg:

class_weight: balanced

max_iter: 2000

gru:

hidden_dim: 64
num_layers: 1
dropout: 0.2
weighted_loss: true

artifacts:

root_dir: artifacts

Model override dosyaları sadece farkları taşıyabilir.

5. Deney akışı

İlk deney

- feature set: mevcut 7 kolon
- lookback: 21
- scaler adayları: standard, robust
- model adayları: logreg, gru
- decision threshold: 0.50
- primary metric: MCC

Sonuç yetersizse

Aynı pipeline'da sadece:

- models.enabled += cnn1d

veya:

- sequence.lookback = 42

veya:

- labeling.threshold_quantile = 0.50

değişir.

Bu, tam istediğin lego mantığıdır.

6. En kritik teknik kurallar

Bunlar kodda ihlal edilmemeli:

- shuffle yok
- random split yok
- final test en başta ayrılacak
- threshold sadece fold train'den hesaplanacak
- scaler sadece fold train'de fit edilecek
- neutral örnekler hedef üretiminden sonra çıkarılacak
- val/test kararları hiçbir seçimde train'e sızmayacak

7. İlk sürüm için minimum yapılacaklar

Kodlamaya başlarken önce şu çekirdek tamamlanmalı:

1. load_data.py
2. preprocess.py
3. splitters.py
4. labeling.py
5. sequence_builder.py
6. model_factory.py
7. trainer.py
8. metrics.py
9. run_experiment.py
10. base.yaml

Bu set tamam olunca sistem çalışır.

8. İlk sprint hedefi

İlk çalışan sürümün çıktısı şu olmalı:

- tek komutla deney çalışsın
- logreg ve gru sonuçları ayrı ayrı çıksın

- CV fold sonuçları kaydedilsin
- en iyi model seçilsin
- final test raporu üretilebilsin
- experiments_summary.csv güncellensin

9. Sonraki pratik adım

Artık doğrudan kod aşamasına geçilebilir.

En doğru üretim sırası:

- önce base.yaml
- sonra veri/preprocess/split/label katmanı
- sonra sequence builder
- sonra logreg baseline
- sonra GRU
- sonra experiment runner

Kod Üretim Sırası

6 Nisan 2026 Pazartesi

02:23

Aşağıdaki sıra, bağımlılıkları bozmadan en az sürtünmeyle kod yazma sırasıdır.

1. configs/base.yaml

İlk dosya bu olmalı. Çünkü geri kalan her şey buradan beslenecek.

İçermesi gereken ana bloklar:

- experiment
- data
- split

- labeling
- sequence
- preprocessing
- decision
- training
- evaluation
- models
- artifacts

Bu dosya hazır olmadan diğer modüllerde sabit değer gömmemelisin.

2. src/utils/config.py

Amaç: YAML'i okuyup tek bir config nesnesi üretmek.

İlk sürüm fonksiyonları:

- load_config(path, overrides=None)
- deep_update(base_dict, override_dict)

Bitti sayılması için:

- base.yaml yüklenebilmeli
- override verilirse ilgili alanlar ezilebilmeli

3. src/utils/seed.py

Amaç: tüm deneyde tek seed kullanmak.

Fonksiyon:

- set_global_seed(seed)

İçeride:

- random
- numpy
- torch
- varsa torch.cuda

Bunu erken yazmak iyi olur, çünkü baştan itibaren her koşu daha tutarlı olur.

4. src/utils/io.py

Amaç: klasör ve artifact yönetimi.

İlk fonksiyonlar:

- ensure_dir(path)
- save_json(obj, path)
- append_row_to_csv(row, path)
- build_artifact_paths(config, experiment_name)

Bu dosya erken yazılırsa sonraki her modül çıktısını düzenli kaydedebilir.

5. src/data/load_data.py

Amaç: veriyi doğru ve güvenli yüklemek.

Fonksiyon:

- load_market_data(path, date_col) -> pd.DataFrame

Yapacakları:

- CSV oku
- date_col parse et
- tarihe göre sırala
- index düzenini temizle

Bitti sayılması için:

- çıktı kronolojik olmalı
- tarih parse hatası vermemeli

6. src/data/preprocess.py

Amaç: eksik veri ve kolon seçimini tek yerde toplamak.

Fonksiyonlar:

- `apply_missing_value_policy(df, method)`
- `select_columns(df, feature_cols, close_col, date_col=None)`

İlk sürüm kuralı:

- `ffill`
- başta kalan NaN satırları sil
- sadece gerekli kolonları bırak

Bu modül bittikten sonra elde temiz bir DataFrame olmalı.

7. src/data/splitters.py

Amaç: zaman serisi split mantığını merkezileştirmek.

Fonksiyonlar:

- `split_dev_test(df, test_ratio)`
- `make_expanding_folds(n_rows, n_folds, val_ratio_within_dev)`

Burada çok dikkat:

- `random` yok
- `shuffle` yok
- `fold`'lar tamamen kronolojik

Bitti sayılması için:

- son %15 test ayrılmalı
- development tarafında 4 expanding fold üretilebilmeli

8. src/data/labeling.py

Amaç: forward return, threshold ve etiket mantığı.

Fonksiyonlar:

- `compute_forward_return(close_series, horizon=1)`
- `compute_threshold(train_returns, method="quantile", quantile=0.40)`
- `make_labels(returns, threshold)`

Çıkış önerisi:

- `bull = 1`
- `bear = 0`
- `neutral = -1`

Bu tasarım iyi olur çünkü neutral sonradan kolay filtrelenir.

Burası bittikten sonra şu garanti edilmeli:

- `threshold` yalnızca train returns ile hesaplanabiliyor
- aynı `threshold` val/test'e uygulanabiliyor

9. src/data/sequence_builder.py

Amaç: lookback'e göre pencere üretmek.

Fonksiyonlar:

- `build_sequences(df, feature_cols, label_series, lookback)`
- `drop_neutral_sequences(X, y, neutral_value=-1)`
- `flatten_sequences(X_seq)`

Buradaki hizalama kuralı net olmalı:

- input = geçmiş lookback adım
- target = t+1 etiketi

Bu modül doğru çalışırsa bütün pipeline'ın kalbi doğru atar.

10. src/evaluation/metrics.py

Amaç: tüm metrikleri tek yerde hesaplamak.

Fonksiyon:

- compute_classification_metrics(y_true, y_pred, y_prob=None)

İçermesi gerekenler:

- MCC
- F1
- Balanced Accuracy
- confusion matrix

Bu dosyayı erken yazmak iyi olur; çünkü her aşamada sonucu hemen ölçebilirsiniz.

11. src/models/logistic_regression.py

İlk çalışan baseline model bu olmalı.

Fonksiyon:

- build_logreg(config)

İçerik:

- class_weight="balanced"
- config'ten max_iter

Neden önce bu?

Çünkü en hızlı çalışan, en kolay debug edilen model bu. Pipeline'ın geri kalanı düzgün mü hemen görürsün.

12. src/models/gru_model.py

İkinci model bu olmalı.

Sınıf:

- GRUClassifier(nn.Module)

İçerik:

- input_size = n_features
- hidden_dim
- num_layers
- dropout
- çıktı tek logit veya probability

Bu aşamada çok karmaşık mimari kurma. İlk sürüm sade olsun.

13. src/models/cnn1d_model.py

İlk sprintte tam doldurmak zorunda değilsin. Ama dosya ve arayüz şimdiden olsun.

Sınıf:

- CNN1DClassifier(nn.Module)

Başlangıçta:

- stub veya minimal skeleton olabilir

Böylece sonra B'den D'ye geçiş kolay olur.

14. src/models/model_factory.py

Amaç: model seçimini tek noktaya toplamak.

Fonksiyon:

- build_model(model_name, config, input_shape)

Örnek:

- logreg
- gru
- cnn1d

Bu dosya lego yapının anahtarıdır. Model seçimi başka yere dağılmamalı.

15. src/training/losses.py

Amaç: class imbalance'ı yönetmek.

Fonksiyon:

- build_loss(config, class_weights=None)

İlk sürüm:

- GRU için weighted BCE veya weighted CE

Bu modül küçük ama gerekli.

16. src/training/callbacks.py

Amaç: early stopping.

Sınıf:

- EarlyStopping

İzleyeceği şey:

- val_mcc

Alanlar:

- patience
- best_score
- counter
- restore_best

17. src/training/trainer.py

Amaç: eğitim mantığını merkezileştirmek.

Fonksiyonlar:

- train_sklearn_model(...)
- train_torch_model(...)
- predict_proba(...)
- predict_labels(...)

Burada iki ayrı akış olacak:

- sklearn tabanlı
- torch tabanlı

Ama dışarıya tek tip çıktı vermeli:

- y_prob
- y_pred
- best_state
- training_summary

18. src/evaluation/reports.py

Amaç: deney çıktısını yazmak.

Fonksiyonlar:

- save_fold_results(...)
- save_experiment_report(...)

- `append_summary_row(...)`

Çıktılar:

- JSON
- summary CSV
- gerekirse confusion matrix görseli

19. `src/run_experiment.py`

En son yazılmalı. Çünkü hepsini orkestre edecek.

İlk sürüm akışı:

1. config yükle
2. seed ayarla
3. veriyi yükle
4. preprocess uygula
5. dev/test split yap
6. fold'ları üret
7. her model için:
 - her scaler için:
 - her fold'da threshold hesapla
 - label üret
 - sequence üret
 - train et
 - validate et
8. en iyi kombinasyonu seç
9. development üzerinde yeniden eğit
10. final testte ölç
11. raporları kaydet

Önerilen gerçek yazım sırası

Bunu aynen uygularsan en az takılırsın:

1. `configs/base.yaml`
2. `src/utils/config.py`
3. `src/utils/seed.py`
4. `src/utils/io.py`
5. `src/data/load_data.py`
6. `src/data/preprocess.py`
7. `src/data/splitters.py`
8. `src/data/labeling.py`
9. `src/data/sequence_builder.py`
10. `src/evaluation/metrics.py`
11. `src/models/logistic_regression.py`
12. `src/models/model_factory.py`
13. `src/training/trainer.py`
14. `src/evaluation/reports.py`
15. `src/run_experiment.py`
16. `src/models/gru_model.py`
17. `src/training/losses.py`
18. `src/training/callbacks.py`
19. `src/training/trainer.py` içine torch akışını genişlet
20. `src/models/cnn1d_model.py`

Sebebi şu:

- önce logreg ile tüm pipeline ayağa kalkar
- sonra GRU eklenir
- en son CNN eklenir

Yani önce omurga, sonra derin model.

Sprint mantığıyla bölersen

Sprint 1

Hedef: çalışan baseline

- config
- utils
- data katmanı
- split
- labeling
- sequence
- metrics
- logistic regression
- run_experiment

Çıktı:

- logreg ile fold sonuçları
- final test raporu

Sprint 2

Hedef: sequence neural model

- GRU
- weighted loss
- early stopping
- torch trainer

Çıktı:

- GRU ile aynı pipeline'da sonuç

Sprint 3

Hedef: genişleme

- CNN1D
- karar eşiği tuning
- ek lookback denemeleri
- ek scaler ve feature deneyleri

Her aşamada kontrol etmen gereken şey

Data katmanı sonrası

- satır sayısı mantıklı mı
- tarih sırası bozulmamış mı
- NaN temizliği doğru mu

Labeling sonrası

- threshold yalnızca train'den mi geliyor
- bull/bear/neutral dağılımı mantıklı mı

Sequence sonrası

- X.shape, y.shape doğru mu
- hedef gerçekten t+1 mi

Split sonrası

- val daima train'den sonra mı
- test en sonda mı

Training sonrası

- fold bazlı MCC geliyor mu
- loglar düzgün yazılıyor mu

En kritik tavsiye

İlk çalışan sürümü **Logistic Regression ile bitir**, sonra GRU'ya geç.

Böyle yaparsan bir sorun çıktığında şu ayrımı kolay yaparsın:

- problem veri/label/split tarafında mı
- yoksa derin model tarafında mı

base.yaml - ilk 5 dosyanın önerilen fonksiyon imzaları

6 Nisan 2026 Pazartesi

02:28

configs/base.yaml

experiment:

name: baseline_bull_bear

seed: 42

notes: "Initial chronological bull/bear classification baseline"

data:

file_path: data/raw/market_data_10y_enriched.csv

date_col: Date

close_col: Nasdaq_Close

feature_cols:

- VIX_Term_Structure
- Yield_Curve
- SKEW_Index
- Risk_Appetite_Ratio
- Crude_Oil
- VWAP_Deviation
- Volume_Momentum

split:

final_test_ratio: 0.15

method: expanding

n_folds: 4

val_ratio_within_dev: 0.10

labeling:

horizon: 1
threshold_method: quantile
threshold_quantile: 0.40
neutral_policy: drop
bull_label: 1
bear_label: 0
neutral_label: -1

sequence:

lookback: 21
flatten_for_tabular: true

preprocessing:

missing_policy: ffill_then_drop_head
scalers:

- standard
- robust

decision:

probability_threshold: 0.50

training:

batch_size: 64
max_epochs: 100
early_stopping_patience: 10
learning_rate: 0.001
monitor_metric: mcc

evaluation:

primary_metric: mcc
secondary_metrics:

- f1
- balanced_accuracy

models:

enabled:

- logreg
- gru

logreg:

class_weight: balanced
max_iter: 2000
solver: lbfgs

gru:

hidden_dim: 64
num_layers: 1
dropout: 0.2
weighted_loss: true

artifacts:

```
root_dir: artifacts
save_fold_reports: true
save_test_predictions: true
save_model_checkpoints: true
```

1) src/utils/config.py

Amaç:

- YAML yüklemek
- override uygulamak
- config doğrulamasını başlatmak

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
from typing import Any, Dict, Optional
```

```
def load_config(config_path: str | Path, overrides: Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
    """Load YAML config and apply optional overrides."""
```

```
def deep_update(base: Dict[str, Any], updates: Dict[str, Any]) -> Dict[str, Any]:
    """Recursively update nested dictionary values."""
```

```
def validate_config(config: Dict[str, Any]) -> None:
    """Validate required config keys and basic value constraints."""
```

2) src/utils/seed.py

Amaç:

- tüm rastgelelik kaynaklarını sabitlemek

```
from __future__ import annotations
```

```
def set_global_seed(seed: int) -> None:
    """Set random seeds for python, numpy, and torch if available."""
```

3) src/utils/io.py

Amaç:

- klasör oluşturma
- json/csv yazma
- artifact path üretme

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
from typing import Any, Dict
```

```
def ensure_dir(path: str | Path) -> Path:
    """Create directory if it does not exist and return Path object."""
```

```
def save_json(data: Dict[str, Any], output_path: str | Path) -> None:
    """Save dictionary as JSON file."""
```

```
def append_row_to_csv(row: Dict[str, Any], csv_path: str | Path) -> None:
    """Append one summary row to a CSV file, creating file if needed."""
```

```
def build_artifact_paths(config: Dict[str, Any], experiment_name: str) -> Dict[str, Path]:
    """Build standard artifact output paths for a given experiment."""
```

4) src/data/load_data.py

Amaç:

- veriyi yüklemek
- tarih parse etmek
- kronolojik sıralamak

```
from __future__ import annotations
```

```
from pathlib import Path
import pandas as pd
```

```
def load_market_data(file_path: str | Path, date_col: str) -> pd.DataFrame:
    """Load CSV, parse date column, sort chronologically, and reset index."""
```

```
def assert_chronological_order(df: pd.DataFrame, date_col: str) -> None:
    """Raise error if dataframe is not sorted increasingly by date column."""
```

5) src/data/preprocess.py

Amaç:

- eksik veri politikası
- kolon seçimi
- temel veri temizliği

```
from __future__ import annotations
```

```
from typing import List
import pandas as pd
```

```
def apply_missing_value_policy(df: pd.DataFrame, method: str) -> pd.DataFrame:
    """Apply missing value handling policy such as ffill_then_drop_head."""
```

```
def select_columns(
df: pd.DataFrame,
feature_cols: List[str],
close_col: str,
date_col: str,
) -> pd.DataFrame:
    """Keep only required date, close, and feature columns."""
```

```
def validate_required_columns(df: pd.DataFrame, required_cols: List[str]) -> None:
    """Raise error if any required columns are missing."""
```

Küçük not

Bu ilk 5 dosyadan sonra doğal sıradaki dosyalar:

- src/data/splitters.py
- src/data/labeling.py
- src/data/sequence_builder.py

Literature review

6 Nisan 2026 Pazartesi
02:32

Rapora koyulabilecek kısa kaynak listesi

1. Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*.
2. Fischer, T., & Krauss, C. (2018). *Deep learning with long short-term memory networks for financial market predictions*. European Journal of Operational Research.
3. Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR.
4. Boughorbel, S., Jarray, F., & El-Anbari, M. (2017). *Optimal classifier for imbalanced data using Matthews Correlation Coefficient metric*. PLOS ONE.
5. Chicco, D., & Jurman, G. (2020). *The advantages of the Matthews correlation coefficient over F1 score and accuracy in binary classification evaluation*.
6. Estrella, A., & Mishkin, F. S. (1996/1998). *The Yield Curve as a Predictor of U.S. Recessions*.
7. Johnson, T. L. (2011). *Equity Risk Premia and the VIX Term Structure*.

En güçlü şekilde desteklenebilecek kısımlar: **GRU seçimi, Adam optimizer, BCE loss, MCC metriği, chronological/expanding validation ve VIX/yield curve gibi finansal feature'lar.**